arXiv:2407.03651v2 [cs.CL] 14 Jul 2024

Evaluating Language Model Context Windows: A "Working Memory" Test and Inference-time Correction

Amanda Dsouza[†] Christopher Glaze[†] Changho Shin[◇] Frederic Sala[†◇]

[†]Snorkel AI
{adsouza, chris.glaze}@snorkel.ai
[◇]University of Wisconsin-Madison
{cshin23, fredsala}@wisc.edu

Abstract

Large language models are prominently used in real-world applications, often tasked with reasoning over large volumes of documents. An exciting development in this space is models boasting extended context capabilities, with some accommodating over 2 million tokens. Such long context model capabilities remain uncertain in production systems, motivating the need to benchmark their performance on real world use cases. We address this challenge by proposing SWiM, an evaluation framework that addresses the limitations of standard tests. Testing the framework on eight long context models, we find that even strong models such as GPT-4 and Claude 3 Opus degrade in performance when information is present in the middle of the context window (lost-in-the-middle effect). Next, in addition to our benchmark, we propose *medoid voting*, a simple, but effective training-free approach that helps alleviate this effect, by generating responses a few times, each time randomly permuting documents in the context, and selecting the medoid answer. We evaluate medoid voting on single document QA tasks, achieving up to a 24% lift in accuracy. Our code is available at https://github.com/snorkel-ai/long-context-eval.

1 Introduction

Real-world applications increasingly use large language models (LLMs) to analyze extensive document collections. Although new models can ingest extended contexts, some accommodating up to 2 million tokens, their actual capabilities — whether they can effectively utilize long context input in practical applications — remain unclear. Evaluating their performance in real-world scenarios is important to enabling these models to operate in production settings.

Evaluating long context capabilities of LLMs has so far been restricted to the popular "needle in a haystack" (NIAH) test [9], its variants [8], or through academic benchmarks [2, 12]. While these may serve as useful starting points for evaluating new model releases, their relevance and applicability to real-world problems are unclear for several reasons:

- **Unrelated Tasks:** The NIAH test often uses unrelated *toy* needles, which do not capture the intricacies of information retrieval in documents, where context and relevance are crucial.
- **Limited Applicability:** Academic benchmarks are often based on datasets that do not represent the nuances and complexities of documents found in real-world scenarios, such as financial reports, legal documents, or customer feedback.
- **Varying model performance:** Model performance can vary significantly depending on the task and data. For instance, experiments by [8] showed that even on standard tasks/datasets, GPT-4 accuracy went from 95-100% on NIAH / retrieval tasks to 79.7% on word extraction tasks to 59.0% on QA tasks.

2 SWiM: Long Context Evaluation Framework for Real-world Tasks

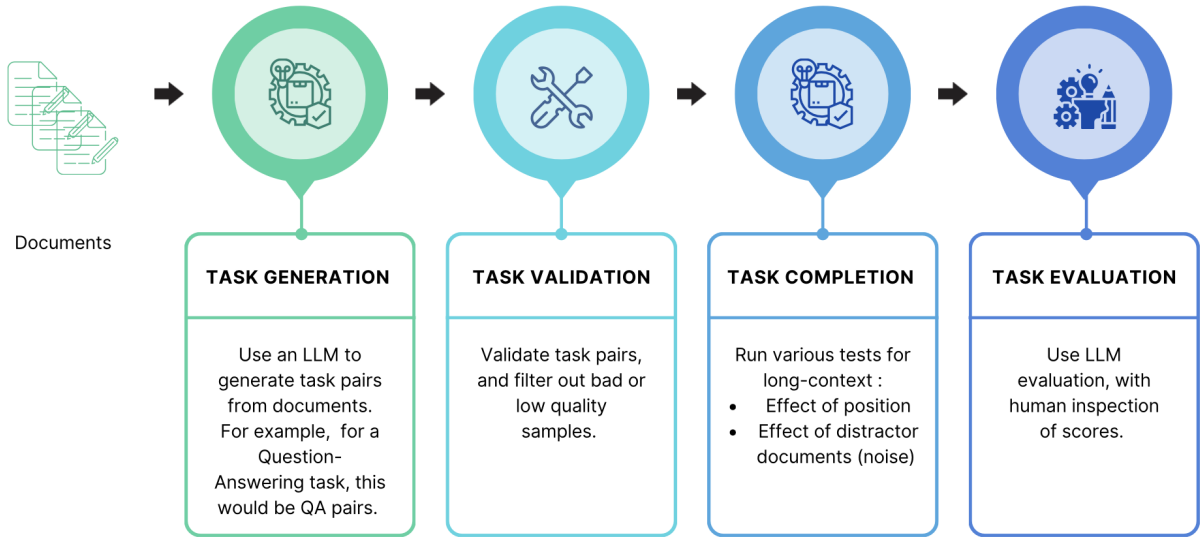

Figure 2: SWiM Framework

To address these problems, we propose Snorkel Working Memory Test (SWiM), an evaluation framework which measures long context capabilities of any language model on use-case specific documents and task pairs. The test and its name are inspired by a wealth of research on human cognition suggesting that short-term memory buffering (as is required in simple recall tasks such as the NIAH) is just one component of a more complex working memory network engaged in using buffered information for more executive functions such as reasoning and goal-directed behavior [1, 5–7].

The SWiM test is executed with a four-step process (Figure 2) of task generation, task validation, task completion and task evaluation. Directly testing on the relevant data and tasks allows a more realistic assessment of a model's long context capabilities for unique applications. Our code is available at https://github.com/snorkel-ai/long-context-eval.

We use SWiM to test eight long context models (including from Open AI, Anthropic, Google, and Mistral), for their effective context length, and the effect of position. Most long context models are ineffective at retrieving information in the middle of the context window (confirming "lost-in-the-middle" effect [10]). To mitigate this effect, we additionally propose a simple algorithmic approach, *medoid voting*, a straightforward yet effective training-free method that improves performance.

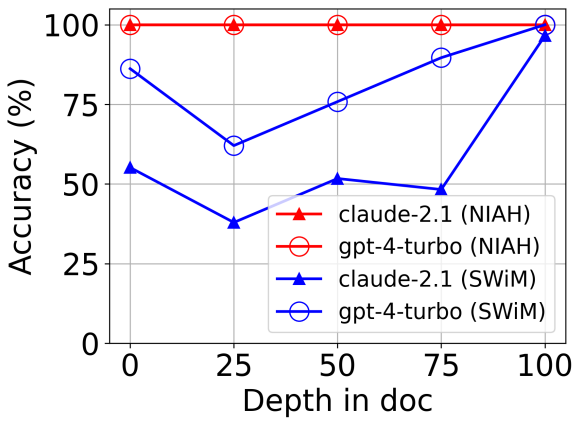

Figure 1: Results of NIAH (answering a synthetic "What is the best thing to do in San Francisco?" needle on Paul Graham essays as the haystack), alongside SWiM on a QA task. GPT-4 and Claude 2.1 obtain perfect scores on the NIAH test, at all document depths. But a more realistic QA task on narrative content using SWiM reveals the typical "lost-in-the-middle" effect.

The framework takes the following four steps.

1. **Task generation**: In order to benchmark long context models on use-case specific data, we automate the creation

of the task using a large language model. We create a Question-Answering task that automatically creates QA pairs on user documents using a large language model.

> **Task generation prompt**
> Ask a factoid question given only the context provided, that can be answered in a few words. Answer the question given the context. Format your result in a JSON object with keys 'question' and 'answer'.
> Context: {context}
> Result:

2. **Task validation**: Using language models to replace human intensive tasks such as dataset creation and evaluation has become commonplace as strong models continue to be released. They are however, prone to errors and without a validation step allows these inaccuracies to trickle down the pipeline, leading to compounding errors and ultimately influencing reported results.

   While prompting and adding guardrails help to some extent, we find that the validation step cannot be circumvented, and a human-in-the-loop approach is best at ensuring good data at the input and output.

3. **Task Completion**: SWiM benchmark supports document QA retrieval, and tests specifically for (a) the effect of position on retrieval accuracy, (b) the effect of context size (the level of noise) on retrieval performance.

   (a) **Effect of position**: [10] showed that models are better at retrieving information from the top or bottom of a long context, while performance greatly reduces when it is contained in the middle. They refer to this phenomenon as the "lost-in-the-middle" effect. In a similar vein, the popular "needle-in-a-haystack" test measures how position affects model performance by injecting a synthetic needle into a set of essays.

   We conduct a similar test to measure how positioning the document within the context window affects retrieving the answer from it. This is similar to the "needle-in-a-haystack" test except over user documents and realistic needles, rather than the synthetic test. This also effectively tests the "lost-in-the-middle" effect, although the experiment setup is slightly different. Liu et al. test on a constant number of 10, 20 and 30 distractor documents (documents that do not contain the answer, but are in the same domain as the answer document), that may not reach the full context window of the model, as well as present the distractor documents in order of decreasing relevance. In contrast, SWiM tests a model on its full context window, with distractor documents shuffled at random. To measure the effect of position on retrieval performance, we follow the procedure in 1. In our experiments, we test varying depths of the true response at 0, 25, 50, 75, and 100% depths.

   (b) **Effect of context size:** Starting with just the answer document in context (0% noise), distractors are successively added, increasing the capacity to 25, 50, 70 and 100% of the context window and model responses generated to test the effective context length of the model in the presence of distractors. To measure the effect of context window used on retrieval performance, we follow the procedure in 2.

4. **Evaluating responses**: We use LLM-as-a-judge to evaluate the responses. On the Single Document QA task, we use the following prompt.

   > **Evaluation prompt**
   > For the question provided, return a JSON object with a 'correct' key as a boolean, if the given answer and gold standard answer are the same in meaning (may be worded differently).
   > Question: {question}
   > Answer: {answer}
   > Gold standard answer: {gold_answer}
   > JSON:

   As with the task generation step, evaluation with a language model is error prone. While the task is a relatively simple one (check if both answers are correct in semantics), there are cases where we find the LLM judge predicts the wrong score. In some cases, the responses contain additional detail, which requires validating the additional

Algorithm 1: SWiM test of document position

Get m documents $\mathcal{D}=\{D_1,D_2,...D_m\}$.[1]
Get (question, answer, answer document index) triplets $(Q_1,A_1,i_1),...(Q_n,A_n,i_n)$.
for $k \leftarrow 1$ to n **do**
  for each position p in (0, 25, 50, 75, 100) **do**
    Get $m-1$ distractor documents $\mathcal{D}_{-k}=\mathcal{D}-D_{i_k}$
    Create context $\mathcal{C}$ with Shuffled $\mathcal{D}_{-k}$ and answer doc D_{i_k} fixed at position p%.
    Generate response to Q_k given context $\mathcal{C}$
  end for
end for

Algorithm 2: SWiM test of context size

Get m documents $\mathcal{D}=\{D_1,D_2,...D_m\}$
Get (question, answer, answer document index) triplets $(Q_1,A_1,i_1),...(Q_n,A_n,i_n)$.
for $k \leftarrow 1$ to n **do**
  $D^*=D_{i_k}$
  for the distractor percentage p% in (0, 25, 50, 75, 100) **do**
    Sample p% documents of $\mathcal{D}$ to construct distractor document set $\tilde{\mathcal{D}}$
    Construct context $\mathcal{C}=\text{Shuffle}(\{D^*\}\cup\tilde{\mathcal{D}})$
    Generate response to Q_k given context $\mathcal{C}$
  end for
end for

Algorithm 3: Medoid voting: SWiM's correction of lost-in-the-middle effect

Input: Document set D, query q, # of permutations k, language model f, embedding model g
Sample random permutations $\sigma_1,...,\sigma_k$ for D
Get LLM ouputs $\hat{y}_i=f([\sigma_i(D),q]),i=1,...,k$
Get embeddings of LLM ouputs $e_i=g(\hat{y}_i),i=1,...,k$
Get medoid in embedding space $\hat{y}=\hat{y}_{i^*}$, where $i^*=\text{argmax}_{i\in\{1,...,k\}}\sum_{j=1}^{k}\cos(e_i,e_j)$.
Return $\hat{y}$

piece of information from the document. Furthermore, we find that LLM judge responses can be inconsistent, even with greedy decoding, giving different scores on the same (or very similar) responses.

3 Medoid Voting: a simple solution to lost-in-the-middle effect

We propose an efficient solution to the lost-in-the-middle effect. Since we know models are generally less effective at retrieving information from some positions than others, it would help if we positioned the answer documents so that they are in the right places. Of course, we do not know what the right place is a priori. A possible solution to this problem is to run the task completion a few times, each time randomly permuting documents in the context, and then use a selection criteria to pick the best response. We consider the medoid response (response with the least dissimilarity to all other responses) in the embedding space as our selection criteria. The detailed procedure is described in Algorithm 3.

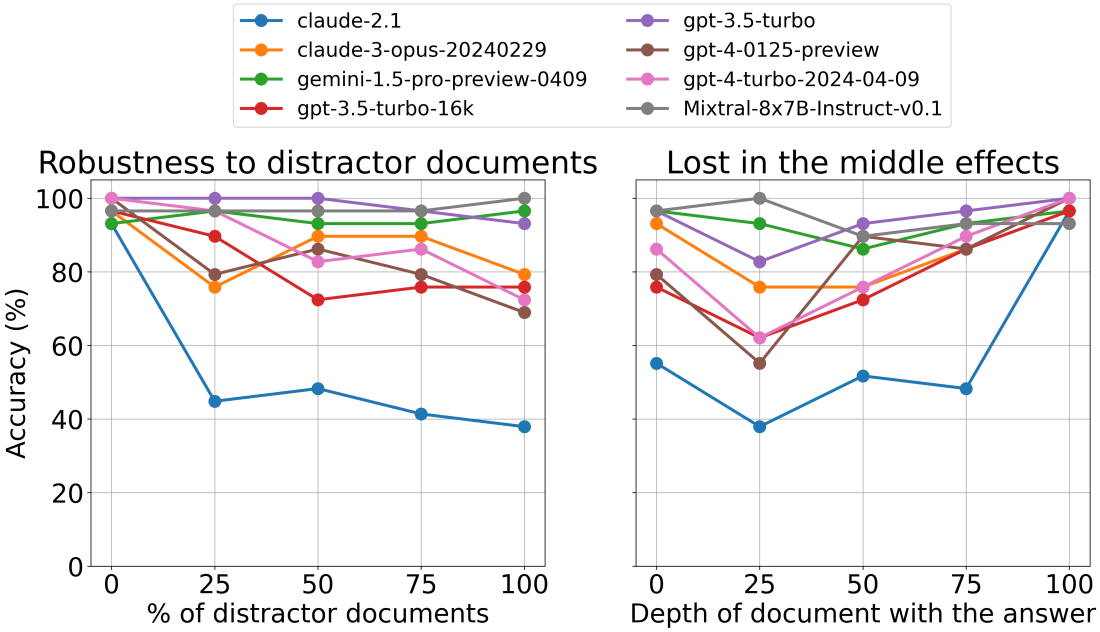

Figure 3: More analyses on LLMs with SWiM framework. (Left) shows that not all models utilize their long context windows effectively, though their context window lengths are enough to include entire documents. (Right) reveals "Lost-in-the-middle" effect is significant and common across many LLMs.

4 Experimental Results

For our experiments, we create QA pairs using GPT-4, from the synthetically generated Huggingface Cosmopedia story_forum dataset [3], treating each instance as a document. We use SWiM to test long context models by Open AI, Anthropic, Google, and Mistral on the Document QA task. Specifically, we evaluate LLMs' robustness to distractor documents and "lost-in-the-middle" effects. Finally, we validate the effectiveness of medoid voting.

4.1 Evaluation on robustness to distractor documents

We evaluate robustness to distractor documents given in the context. This investigates how well LLMs can extract the necessary information without being distracted to irrelevant context.

Setup. We test single document QA performance with an increasing number of documents that do not contain the answer (distractor documents). Starting with only the answer document, distractors are added to fill up the context window to 25, 50, 75 and 100% of its capacity[2], and the set is shuffled within the context window for response generation.

Results. Figure 3 (left) shows the experiment results. Unsurprisingly, as the number of distractors increase, performance degrades. But this degradation is not uniform across models. Among models with a long context window (1M tokens), Gemini-1.5-Pro does extremely well to handle noise. It is also unsurprising that smaller context length models (such as GPT-3.5 Turbo and Mistral-8x7B-Instruct) are more effective at using their context lengths compared to larger context length ones. Analyzing incorrect responses, we find that many of these are due to how documents are positioned in the context, which we discuss next.

[2]For simplicity we use tiktoken to count tokens. For non OpenAI models, this means that we use slightly lower context sizes than the reported size.

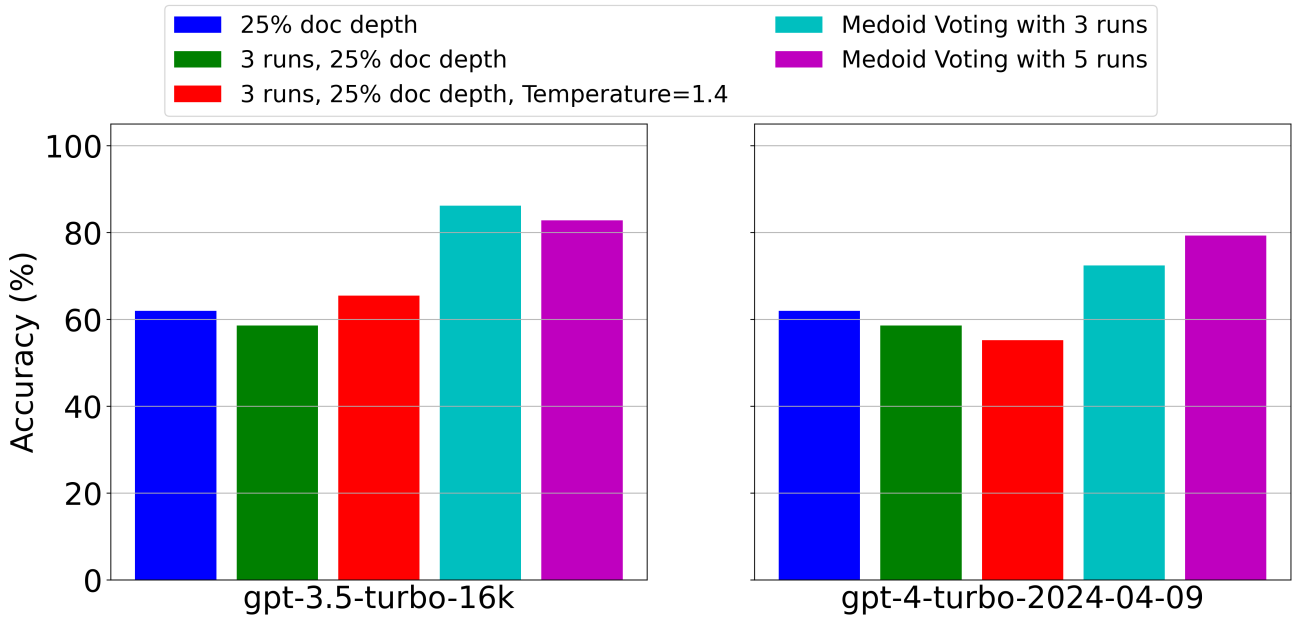

Figure 4: Medoid voting can easily smooth out the "lost-in-the-middle" effect.

4.2 Evaluation on robustness to document position

We evaluate long-context LLMs' robustness to document position. It is well known that position affects the retrieval performance of long context LLMs, but the effect varies depending on benchmarks and models [10, 12]. We expect that lost-in-the-middle effect is common to long context models.

Setup. We use SWiM to test eight models on the single document QA task. We place the answer document at the 25, 50, 75 and 100% positions in the context length and fill up the rest of the context window with randomly shuffled distractor documents. Note that since we first compute the number of tokens of documents to fit the context window of a model, the number of documents used for a model may differ based on the model's context window size. Across the position tests for a given model, the set of documents is kept fixed, to reduce confounding effects of distractors on the performance.

Results. Figure 3 (left) shows the experiment results. Most models exhibited a degradation in performance at the 25% $\sim$ 75% depth and showed their best performance when the answer is located at 0% depth or 100% depth. The observed performance degradation of long context models in the middle poses significant implications for real world applications.

Additionally, this result implies that the NIAH test, while a good way to quickly test new models, is often not a good indicator of real world performance. This is evident in Claude-2.1's results, where NIAH results showed strong performance when the needle was at the top and bottom of the context, but we do not find the same behavior when the document is at the top of a long context.

4.3 Medoid voting

Setup. We tested medoid voting on two models that observed the lost-in-the-middle effect, GPT-4-Turbo and GPT-3.5-Turbo-16k. To isolate the impact of the medoid voting, we conducted a control experiment that generates multiple runs keeping document position constant at the unfavorable 25% document depth, with model stochasticity as the only source of variation. We used temperatures of 0.7 (default) and 1.4 (a high temperature to induce higher variance in responses). This lets us test the specific effect that varying position has over other forms of variation in

generating the final response.

Results. Figure 4 shows the results. We found medoid voting to be effective, even with as few as 3 runs. Results show the medoid voting method surpasses performance over both the baseline and controls, on both models; a 17.3 point lift on GPT-4-Turbo, and 24.2 point lift on GPT-3.5-Turbo-16k. This underscores medoid voting's superiority—a straightforward, training-free strategy that can be used to enhance performance.

We additionally found that medoid voting might compromise the best case performance by averaging effect. However, we easily address this issue by filtering absence outputs ("No information found in context"), assuming the proper information can be found in the context. Figure 5 shows the analysis and further improvement by absence filtering.

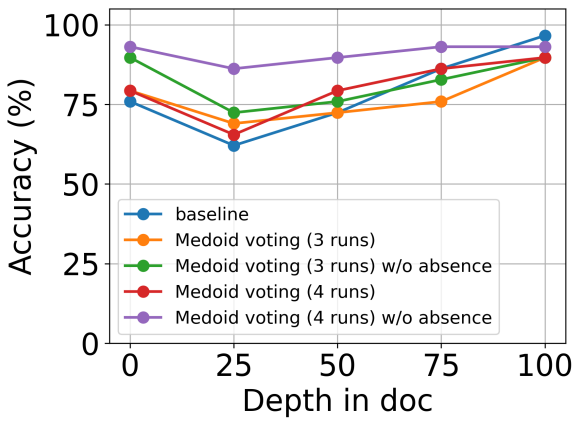

Figure 5: Voting accuracy depending on the depth of document with answer

5 Related Work

Existing long context benchmarks. As the effectiveness of in-context learning [4] has been studied, benchmarks to evaluate long-context capacity of LLMs have been actively developed. Long Range Arena [11] evaluates LLMs with the context length from 1K to 16K tokens, including various data types and modalities such as text, images, and mathematical expressions. The needle in a haystack [9] (NIAH) test evaluates LLMs' capability to retrieve answers from the given context (essays). RULER benchmark [8] expands upon the NIAH test to include more variations such as multi-hop tracing and aggregation. LongBench [2] provides a benchmark with single-doc QA, multi-doc QA, summarization, few-shot learning, code completion, and synthetic tasks in English and Chinese. ∞Bench [12] provides datasets and pipelines to evaluate LLMs with 100K+ context in English and Chinese. While these have been useful for evaluating long-context capability of LLMs, they lack customizability—their tasks might not be directly relevant to the problems encountered in business applications. Our framework, instead, provides an end-to-end customizable evaluation pipeline.

6 Conclusion

We propose the SWiM framework, which enables users to create a personalized benchmark to evaluate long context models on their data with their tasks. Experimental results suggest that SWiM can identify patterns of errors that NIAH cannot, so we strongly recommend usage of SWiM prior to any development work for specific applications. However, there is more experimental work required to draw general conclusions about the language models used in research reported here. While we saw that some models such as Gemini-1.5-Pro do extremely well handling long contexts in the setting of single document QA, they may be less effective in more complex scenarios. These complex scenarios include relevant documents that have been combined with many other similar documents; tasks requiring reasoning over multiple documents; and tasks requiring citations to specific sources within documents. Future development of the SWiM framework should address these complex scenarios with more fine grained evaluations (e.g. including hallucination detection), and on a wider array of tasks.

Acknowledgments

We thank Paroma Varma and Armin Parchami for their helpful feedback and discussion.

References

[1] Alan D Baddeley and Graham Hitch. Working memory. 8:47–89, 1974. ISSN 0079-7421. doi: https://doi.org/10.1016/S0079-7421(08)60452-1. URL https://www.sciencedirect.com/science/article/pii/S0079742108604521.

[2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

[3] Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. Cosmopedia, 2024. URL https://huggingface.co/datasets/HuggingFaceTB/cosmopedia.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[5] Nelson Cowan. What are the differences between long-term, short-term, and working memory? In *"Progress in Brain Research"*, volume 169, pages 323–338. Elsevier, 2008.

[6] Mark D'Esposito and Bradley R Postle. The cognitive neuroscience of working memory. *Annual Reviews of Psychology*, 66:115–142, 2015.

[7] Randall W Engle. Working memory capacity as executive attention. *Current Directions in Psychological Science*, 11(1):19–23, 2002. doi: 10.1111/1467-8721.00160.

[8] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[9] Gregory Kamradt. Llmtest needle in a haystack - pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.

[10] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[11] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. *arXiv preprint arXiv:2011.04006*, 2020.

[12] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, et al. ∞ bench: Extending long context evaluation beyond 100k tokens. *arXiv preprint arXiv:2402.13718*, 2024.